ESP32 QEMU Bare-Metal Lab

Environment Setup, QEMU Testing, and Physical Flashing Guide

Repository: github.com/akramnemri/esp32-qemu-lab.git

This version keeps the original workflow, adds a physical flashing path, moves the WSL2 USB instructions into the WSL section, replaces the old checklist with a clearer troubleshooting section, and tightens the visual design for easier reading.

---

1. Environment Options

Choose one of the three methods below to get a working Linux environment. All methods converge to the same Common Setup steps.

Method	Best for	Requirements
WSL2	Windows 10/11 users	Windows 10 v2004+ or Windows 11. WSL2 enabled. Ubuntu 22.04/24.04 installed via Microsoft Store.
Dual Boot	Users who prefer native Linux	Existing Ubuntu 22.04/24.04 LTS (or Debian-based) installation with 4 GB+ RAM and 25 GB+ free disk space.
Virtual Machine	Any OS (Windows, macOS, Linux hosts)	VM software installed (VirtualBox, VMware, UTM, or Hyper-V). Ubuntu 22.04/24.04 ISO. 4 GB+ RAM allocated, 25 GB+ disk.

Important

QEMU for ESP32 is a user-mode/system emulator and does not require nested virtualization (KVM). It will run fine inside a VM.

2. Option A: WSL2 Setup (Windows 10/11)

This is the recommended method for Windows users. WSL2 provides a full Linux kernel with excellent performance and seamless Windows integration.

2.1 Enable and Install WSL2

```
Open Windows PowerShell as Administrator and run:
wsl --install
wsl --update
wsl --set-default-version 2
```

Time & Size: ~5-10 minutes depending on internet speed. WSL2 + Ubuntu download: ~1.5 GB.

Reboot your computer if prompted. After reboot, launch Ubuntu from the Start menu and complete the initial user setup (username and password).

2.2 Update Ubuntu Packages

```
sudo apt update && sudo apt upgrade -y
```

Time & Size: ~3-5 minutes. Updates vary based on package age; typically 200-500 MB of downloads.

2.3 WSL2 Specific Notes

- Ensure your Windows build is version 19041 (20H1) or later for full WSL2 support.
- If you encounter virtualization errors, enable Virtual Machine Platform and Windows Subsystem for Linux in Windows Features.
- For file system performance, keep project files inside the Linux home directory (/home/<user>/) rather than Windows drives (/mnt/c/).

2.4 WSL2 USB Setup for Physical Flashing

WSL2 does not natively expose USB serial devices. Install usbipd on Windows to attach the ESP32 to WSL2 when you want to flash real hardware.

```
Install usbipd on Windows (PowerShell as Administrator):
winget install --interactive --exact dorssel.usbipd-win

Attach ESP32 to WSL2:
usbipd list
usbipd bind --busid <BUSID>
usbipd attach --wsl --busid <BUSID>

Inside WSL2, grant port permissions:
sudo chmod 666 /dev/ttyUSB0
# Permanent fix: add your user to the dialout group
sudo usermod -a -G dialout $USER
```

Time & Size: usbipd install takes about 2-3 minutes. Attaching the board takes less than 1 minute per reconnect. Re-attach after each USB disconnect or Windows reboot. Log out and back in after adding your user to the dialout group.

3. Option B: Dual Boot (Native Linux)

This section assumes you already have a dual-boot Linux installation. If you need to install Linux, refer to the Ubuntu installation guide. The following requirements must be met for the project to work.

3.1 Prerequisites

- OS: Ubuntu 22.04 LTS or 24.04 LTS (or any Debian-based distribution)
- RAM: Minimum 4 GB (8 GB recommended for comfortable multitasking)
- Disk Space: Minimum 25 GB free (35 GB recommended to accommodate ESP-IDF, toolchains, and build artifacts)
- Internet: Stable connection for downloading ESP-IDF and toolchain packages

  Verify your installation:

```
lsb_release -a
free -h
df -h
```

Time & Size: No additional download time if already installed. Verify disk space: ESP-IDF + toolchain requires ~8-10 GB.

3.2 Update Packages

```
sudo apt update && sudo apt upgrade -y
```

Time & Size: ~3-5 minutes. Update size varies; typically 100-500 MB.

4. Option C: Virtual Machine

This section assumes you already have VM software installed and a Linux VM created. If you need to set up a VM, use VirtualBox, VMware Workstation, UTM (macOS), or Hyper-V. The following requirements must be met.

4.1 VM Requirements

- VM Software: VirtualBox 7.0+, VMware Workstation 17+, UTM, or Hyper-V
- Guest OS: Ubuntu 22.04 LTS or 24.04 LTS (64-bit)
- Allocated RAM: Minimum 4 GB (6-8 GB recommended)
- Allocated Disk: Minimum 25 GB dynamically allocated (40 GB recommended)
- CPU: 2+ cores allocated to the guest
- Network: NAT or Bridged adapter with internet access

> **Note**
> Nested virtualization (KVM) is NOT required.
> QEMU for ESP32 runs in user-mode and does not need hardware acceleration.

Time & Size: Ubuntu ISO download: ~5 GB. VM creation and OS installation: ~20-30 minutes.

4.2 Update Packages

```
sudo apt update && sudo apt upgrade -y
```

Time & Size: ~3-5 minutes. Update size varies; typically 100-500 MB.

4.3 VM-Specific Tips

- Shared Clipboard: Enable bidirectional clipboard in VM settings for easy copy-paste between host and guest.

- Shared Folders: Mount a shared folder if you want to transfer files between host and VM.
- Display: QEMU runs with -nographic, so no GUI is needed inside the VM for this project.
- Snapshots: Take a VM snapshot after completing the Common Setup to avoid repeating installations.

5. Common Setup: Install ESP-IDF & Tools

These steps are identical regardless of which environment method you chose (WSL2, Dual Boot, or VM).

5.1 Install System Dependencies

```
sudo apt install -y git wget flex bison gperf cmake ninja-build ccache \
  libffi-dev libssl-dev dfu-util python3 python3-pip python3-venv \
  unzip xz-utils file make
```

Time & Size: ~5-8 minutes. Package downloads: ~300-500 MB.

5.2 Download ESP-IDF v5.2

```
mkdir -p ~/esp
cd ~/esp
git clone -b v5.2 --recursive https://github.com/espressif/esp-idf.git
```

Time & Size: ~10-15 minutes. Repository size: ~2.5 GB (includes submodules for all chip families).

5.3 Install ESP-IDF (Toolchain + QEMU)

```
cd ~/esp/esp-idf
./install.sh esp32
```

Time & Size: ~15-25 minutes. Toolchain + QEMU download: ~6-8 GB total. Total ESP-IDF footprint after install: ~8-10 GB.

> **Note**
> If the installation seems to hang, it is usually downloading large toolchain archives.
> Be patient and ensure a stable internet connection.

5.4 Make Environment Permanent

```
echo 'export IDF_PATH="$HOME/esp/esp-idf"' >> ~/.bashrc
echo '. $IDF_PATH/export.sh' >> ~/.bashrc
source ~/.bashrc
```

Time & Size: <1 minute. No additional downloads.

5.5 Verify Installation

```
xtensa-esp-elf-gcc --version
idf.py --version
qemu-system-xtensa --version
```

Time & Size: <1 minute. No downloads.

All three commands should print version numbers. If any command is missing, manually re-export:

```
source ~/esp/esp-idf/export.sh
```

6. Build & Run the Project

Now that the environment is ready, clone the repository and build the firmware.

6.1 Clone the Repository

```
cd ~
git clone https://github.com/akramnemri/esp32-qemu-lab.git
cd esp32-qemu-lab
```

Time & Size: <1 minute. Repository size: <5 MB (source code only).

6.2 Set Target and Build

```
idf.py set-target esp32
idf.py build
```

Time & Size: ~3-5 minutes (first build). Build artifacts: ~500 KB - 2 MB. Subsequent builds are faster due to caching.

Successful build generates:

```
build/bootloader/bootloader.bin
build/partition_table/partition-table.bin
build/main.bin
```

6.3 Create the QEMU Flash Image

```
esptool.py --chip esp32 merge_bin \
  -o build/qemu_flash.bin \
  --flash_mode dio \
  --flash_size 4MB \
  --flash_freq 40m \
  0x1000 build/bootloader/bootloader.bin \
  0x8000 build/partition_table/partition-table.bin \
  0x10000 build/main.bin


truncate -s 4M build/qemu_flash.bin
```

Time & Size: <1 minute. Flash image size: exactly 4 MB (4,194,304 bytes).

6.4 Run in QEMU

```
qemu-system-xtensa -nographic \
  -machine esp32 \
  -drive file=build/qemu_flash.bin,if=mtd,format=raw \
  -serial mon:stdio \
  -global driver=timer.esp32.timg,property=wdt_disable,value=true
```

Time & Size: QEMU boots in ~2-3 seconds. No additional downloads.

7. Expected Output

After the ESP32 boot messages, the terminal should display the following pattern repeating indefinitely:

```
=== ESP32 QEMU Bare-Metal ===
Pattern: 2 quick -> 2 long -> 2s on -> 2s off
================================
CYCLE START
2 quick done
2 long done
hold done
CYCLE START
2 quick done
2 long done
hold done
...
```

How to stop QEMU

Stop emulator: Press Ctrl+A then X
Enter monitor: Press Ctrl+A then C (type 'quit' to exit)

8. Physical Hardware Path: Flash to Real ESP32

This path flashes the firmware to a physical ESP32 development board. The onboard LED on GPIO2 will blink, and UART output appears on the serial monitor.

8.1 Prerequisites

- Board: ESP32-DevKitC, ESP32-WROOM-32, or any ESP32 dev board with onboard LED on GPIO2
- Cable: USB cable with data lines (not charge-only)
- Drivers: USB-to-UART bridge drivers (CP210x, CH340, or FTDI) installed on Windows

8.2 Disable Watchdogs in sdkconfig

The ESP32's Task and Interrupt Watchdogs can trigger resets during blocking delay loops. Disable them in sdkconfig:

```
sed -i 's/^CONFIG_ESP_INT_WDT=.*/# CONFIG_ESP_INT_WDT is not set/' sdkconfig
sed -i 's/^CONFIG_ESP_INT_WDT_TIMEOUT_MS=.*/# CONFIG_ESP_INT_WDT_TIMEOUT_MS is not set/' sdkconfig
sed -i 's/^CONFIG_ESP_INT_WDT_CHECK_CPU1=.*/# CONFIG_ESP_INT_WDT_CHECK_CPU1 is not set/' sdkconfig
sed -i 's/^CONFIG_ESP_TASK_WDT_INIT=.*/# CONFIG_ESP_TASK_WDT_INIT is not set/' sdkconfig


grep -E "CONFIG_ESP_INT_WDT|CONFIG_ESP_TASK_WDT" sdkconfig
```

All watchdog-related lines should show # ... is not set.

8.3 Build and Flash

```
cd ~/esp32-qemu-lab
```

```
idf.py set-target esp32
idf.py build
idf.py -p /dev/ttyUSB0 flash monitor
```

Time & Size: ~5-8 minutes total (build + flash). Flash takes ~5-10 seconds at 460800 baud.

To re-flash

Put the board into download mode: Hold BOOT, press EN (RESET), release BOOT, then run the flash command.
Exit monitor: Press Ctrl+]

Important

For physical flashing in WSL2, use the USB steps in Section 2.4 first.
Inside WSL2, the device usually appears as /dev/ttyUSB0 after attachment.

9. Expected Output

After the ESP32 boot messages, the terminal should display the following pattern repeating indefinitely:

```
=== ESP32 QEMU Bare-Metal ===
Pattern: 2 quick -> 2 long -> 2s on -> 2s off
================================
CYCLE START
2 quick done
2 long done
hold done
CYCLE START
2 quick done
2 long done
hold done
...
```

QEMU: Output appears in the terminal window. No LED (emulation only). Physical Hardware: Onboard LED on GPIO2 blinks the pattern: 2 quick -> 2 long -> 2s on -> 2s off -> repeat. UART output appears at 115200 baud.

10. Troubleshooting

Quick fixes for common issues:

Problem	Solution
xtensa-esp-elf-gcc not found	Run source ~/esp/esp-idf/export.sh and confirm ~/.bashrc contains the export lines.
idf.py not found	IDF_PATH is missing or incorrect. Re-run the permanent export step in Section 5.4.

qemu-system-xtensa missing	Re-run ./install.sh esp32 inside ~/esp/esp-idf.
Nothing prints in QEMU	Confirm the flash image was built with esptool.py merge_bin and padded to 4MB. Ensure -serial mon:stdio is used.
QEMU serial/monitor conflict	Use -serial mon:stdio (already included in Section 6.4).
Could not open /dev/ttyUSB0	Run usbipd attach --wsl --busid <BUSID> from Windows PowerShell. Check ls /dev/ttyUSB* in WSL2.
Port busy / Resource unavailable	Previous monitor is still running. Press Ctrl+] to exit, or run sudo chmod 666 /dev/ttyUSB0.
WDT reset on physical hardware	Disable watchdogs in sdkconfig (Section 8.2). Ensure vTaskDelay is used, not nop spin loops.
No LED blink on physical hardware	Verify GPIO_FUNC2_OUT_SEL_CFG_REG is at 0x3FF44538 (not 0x3FF44554). Check MCU_SEL = 2 in IO MUX.
Build fails with linker errors	Ensure you ran idf.py set-target esp32 before building. Clean with idf.py fullclean and rebuild.